This is a good opportunity for some small-scale design. You want each task to have an order, the project to display the tasks in order, and new tasks to come in at the end of the list.

You can handle the first part, giving tasks an order, with a Rails migration:

```
$ rails generate migration add_order_to_tasks
```

I'm calling the new attribute project_order to avoid confusion with the SQL order statement:

integration/02/db/migrate/20170626013611_add_order_to_tasks.rb

```
class AddOrderToTasks < ActiveRecord::Migration[5.1]
  def change
    change_table :tasks do |t|
      t.integer :project_order
    end
  end
end
```

```
$ rake db:migrate
```

You can make products automatically return tasks in order by going into the Project class definition and changing the declaration of the relationship to this:

```
has_many :tasks, -> { order "project_order ASC" }, dependent: :destroy
```

You don't need a test because it's part of the framework.

For the tests to work, you also need to add the project_order to the let statements in the spec:

integration/02/spec/system/add_task_spec.rb

```
let!(:task_1) { create(
  :task, project: project, title: "Search Sky", size: 1, project_order: 1) }
let!(:task_2) { create(
  :task, project: project, title: "Use Telescope",
         size: 1, project_order: 2) }
```

Giving a new task in an order adds some logic. There are several ways to do this. Here are a few examples:

- You could make the logic an after_save callback on Task, which would be invoked automatically when the task is saved.
- You could make it a method on Project to ask the project for the next order.
- You could create a Task workflow object similar to the CreatesProject object you built earlier that would query the Project to get the new order.